Marburg University

Programmierpraktikum Einführung in Machine Learning

AGs
Becker (Deep Decision Support Systems),
Braun (Natural Language Processing),
Ewerth (Multimodal Modelling and Machine Learning),
Seifert (Explainable Artificial Intelligence)

Daniel Schneider

31.08. - 01.09.2026

Agenda

1. Introduction
2. Clustering
3. Decision Trees
4. Artificial Neural Networks
5. Packages
   - Scikit Learn
   - Tensorflow

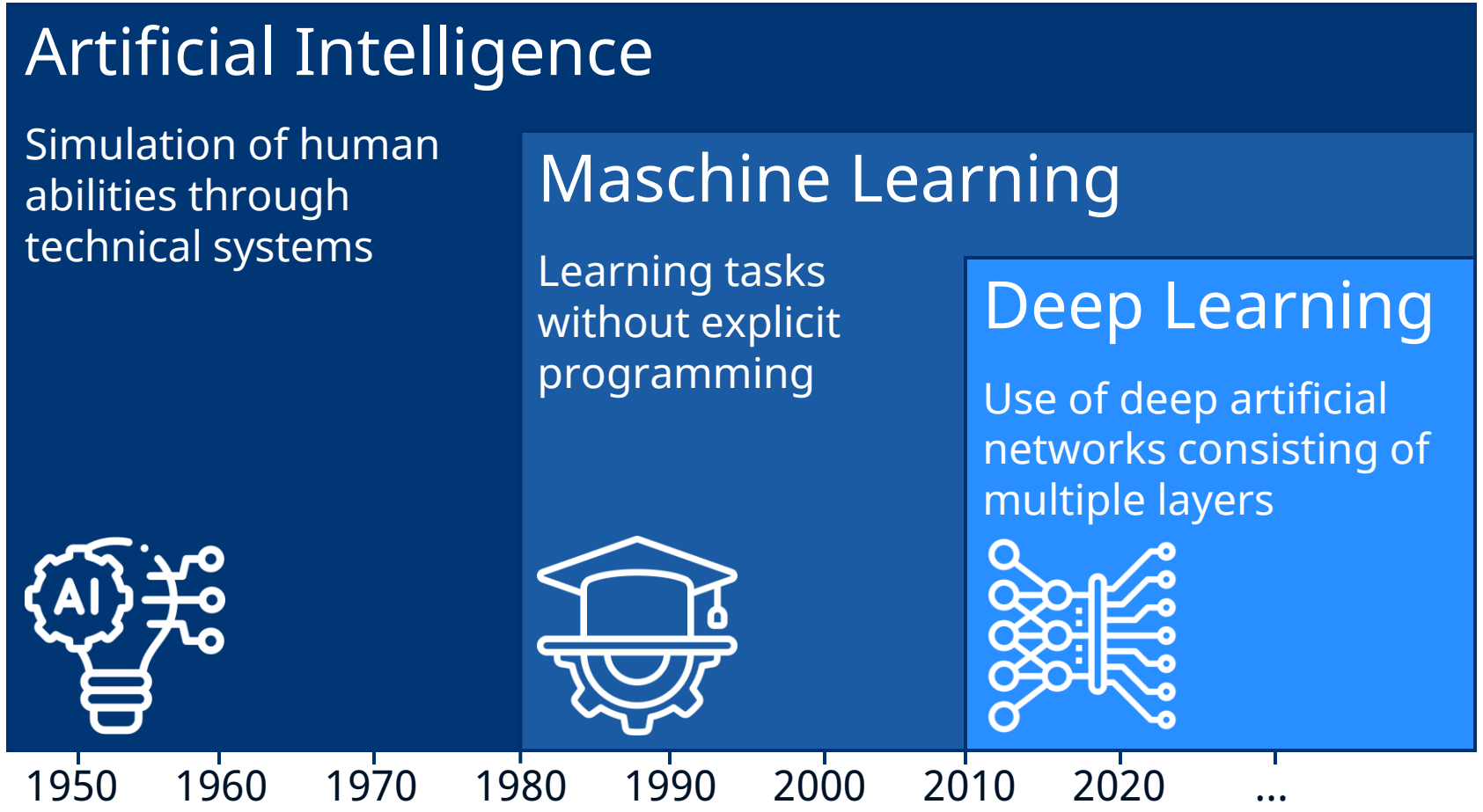
Introduction

What is Machine Learning?

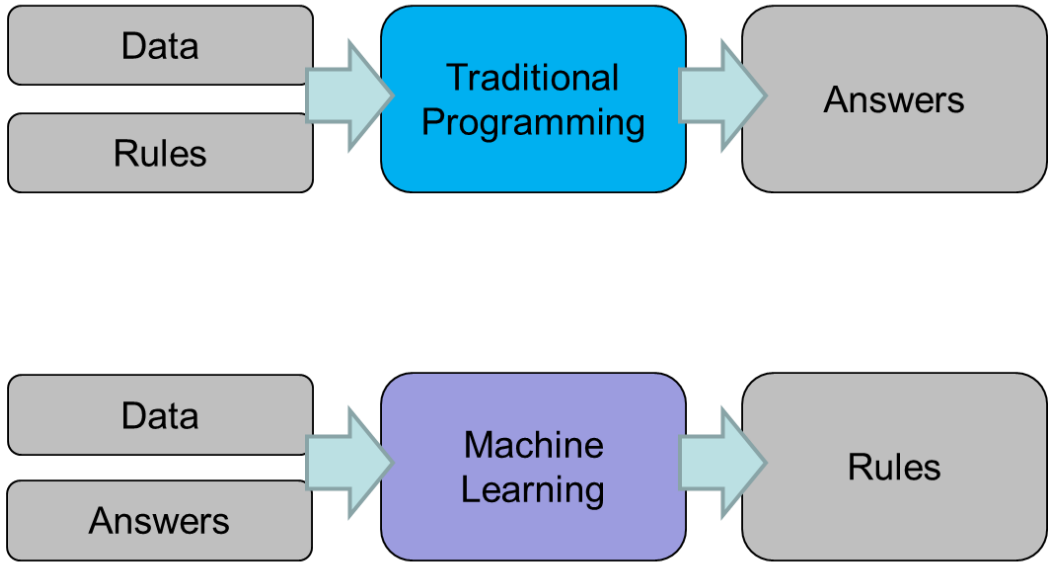
- Arthur Samuel (1959):
  Machine Learning: Field of study that gives computers the ability to **learn** without being explicitly programmed.

- Tom Mitchell (1998):
  Well-posed Learning Problem: A computer program is said to learn from **experience** E with respect to some **task** T and some **performance measure** P, if its performance on T, as measured by P, improves with experience E.

AI, Machine Learning and Deep Learning



Machine learning



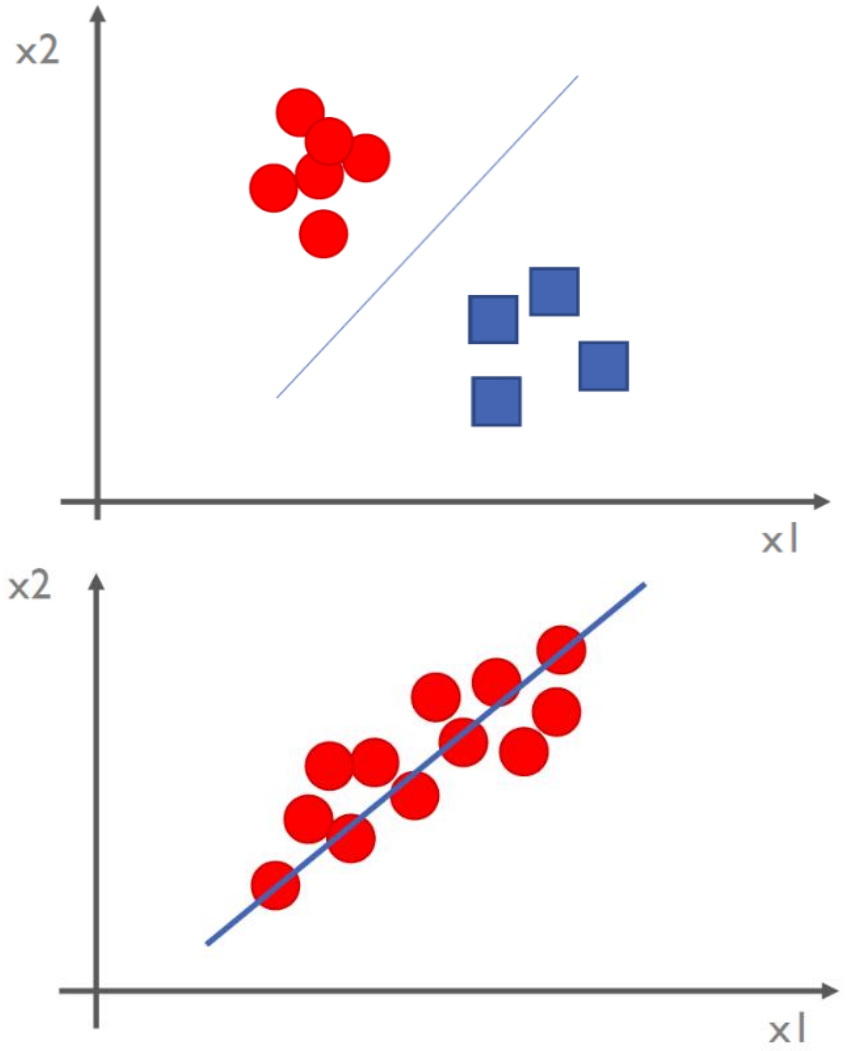
- Automatic search for hypotheses to explain data
- Algorithms more accurate than "human" rules

Supervised learning

- **Data**
  - Relations are known
  - **Labeled** training data

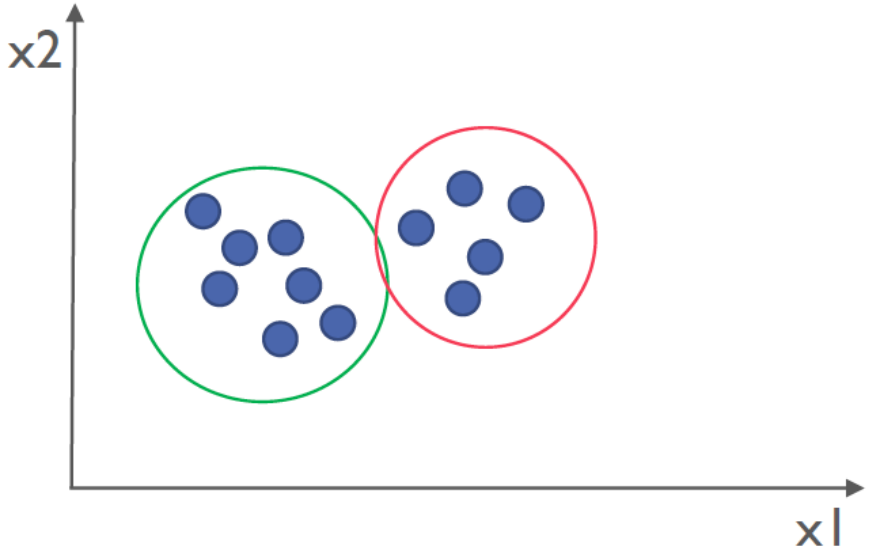
- **Goal:** Learning a **prediction model**

- **Classification**:
  - Categorical target variable (discrete output value)
- **Regression**:
  - Metric target variable (continuous output value)



Unsupervised learning

- **Data**
  - No ground truth information given
- **Goal:** Finding hidden structures in unlabeled data
- **Examples**
  - Clustering
  - Dimension reduction



Clustering

Cluster analysis

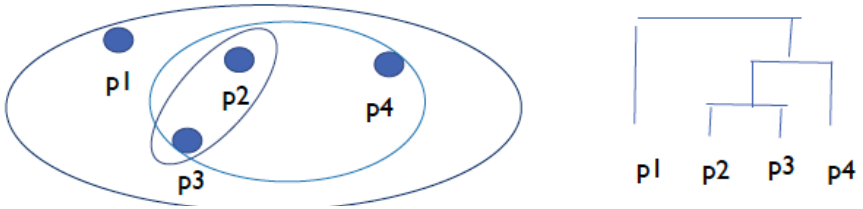
- **Goal:** Allocation of data points to clusters with
  - Large **inter-cluster diversity**
    - differences between elements of different clusters should be as large as possible
  - High **intra-cluster similarity**
    - elements of one cluster should be as similar as possible
- No grouping of attributes, but grouping of the collected objects (e.g. persons) based of their specific feature values
- Calculation of dissimilarities, e.g. with the Euclidean distance

$$d_{Eucidean}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

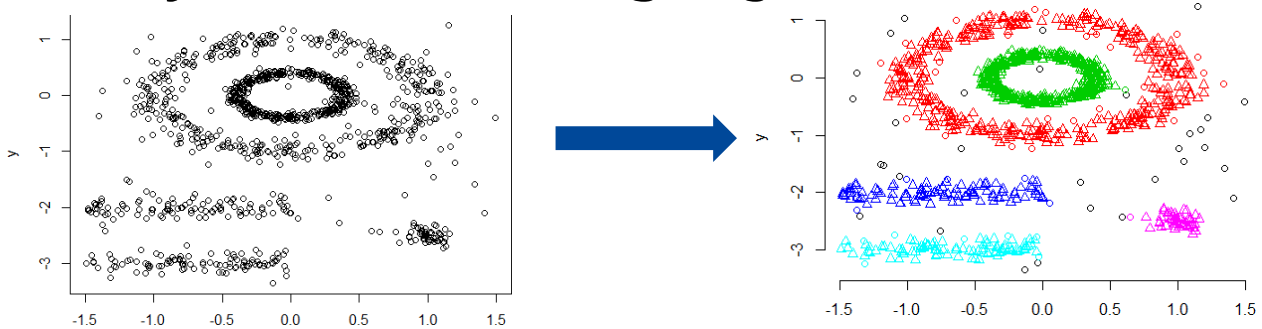
Types of clustering

- **Partitional Clustering** (e.g. K-means)
  - Classification into non-overlapping clusters

- **Hierarchical Clustering**
  - Set of nested clusters, organized as a hierarchical tree

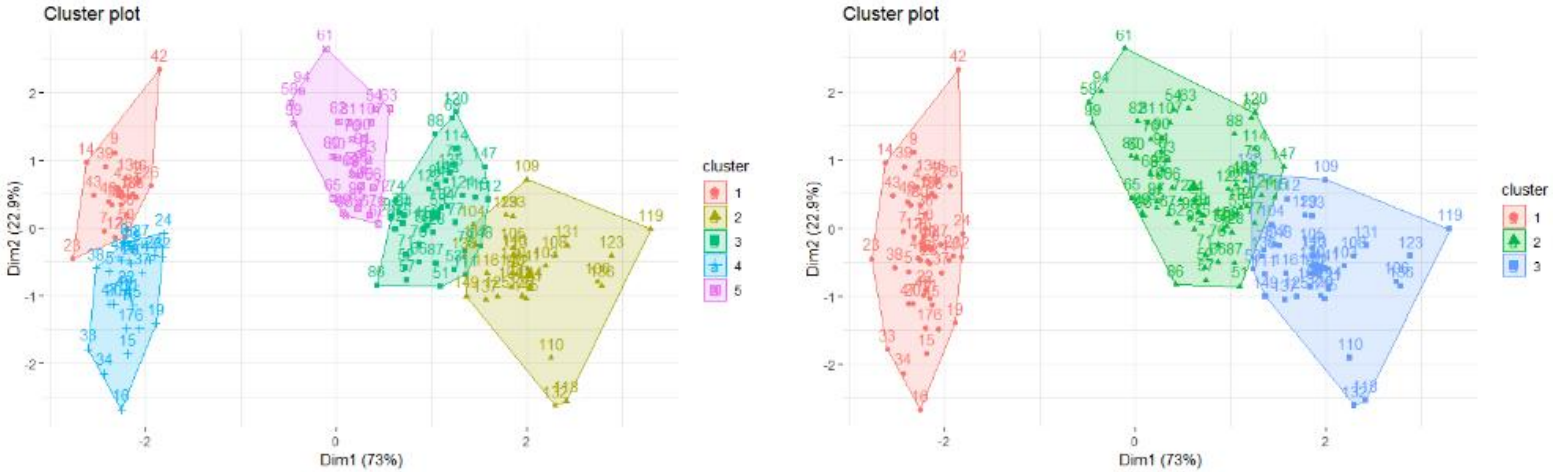


- **Density-based Clustering** (e.g. DBSCAN)



K-Means

- This algorithm assumes that the number of groups to be found is already known
  - Specification of the number of clusters k required
  - Can lead to problems



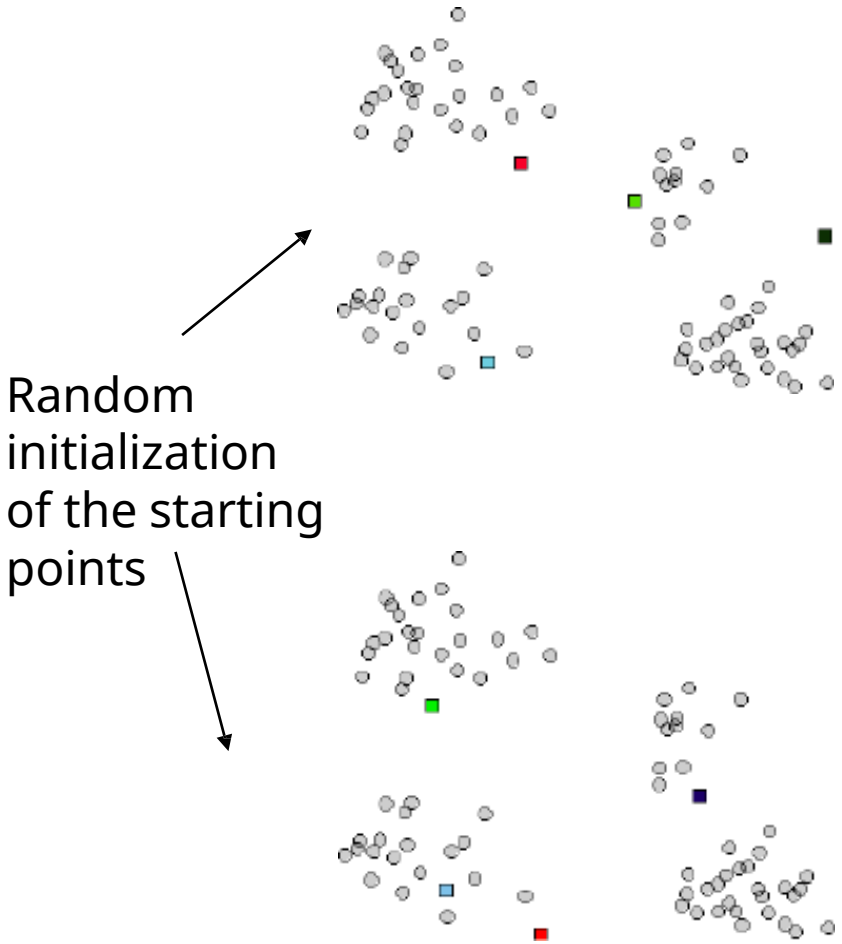
- Goal: Minimize the sum of squared distances from the cluster centers
  - Optimization function: $J = \sum_{i=1}^{k} \sum_{p \in C_i} d(p, m_{C_i})^2$

K-Means

- **Iterative algorithm**

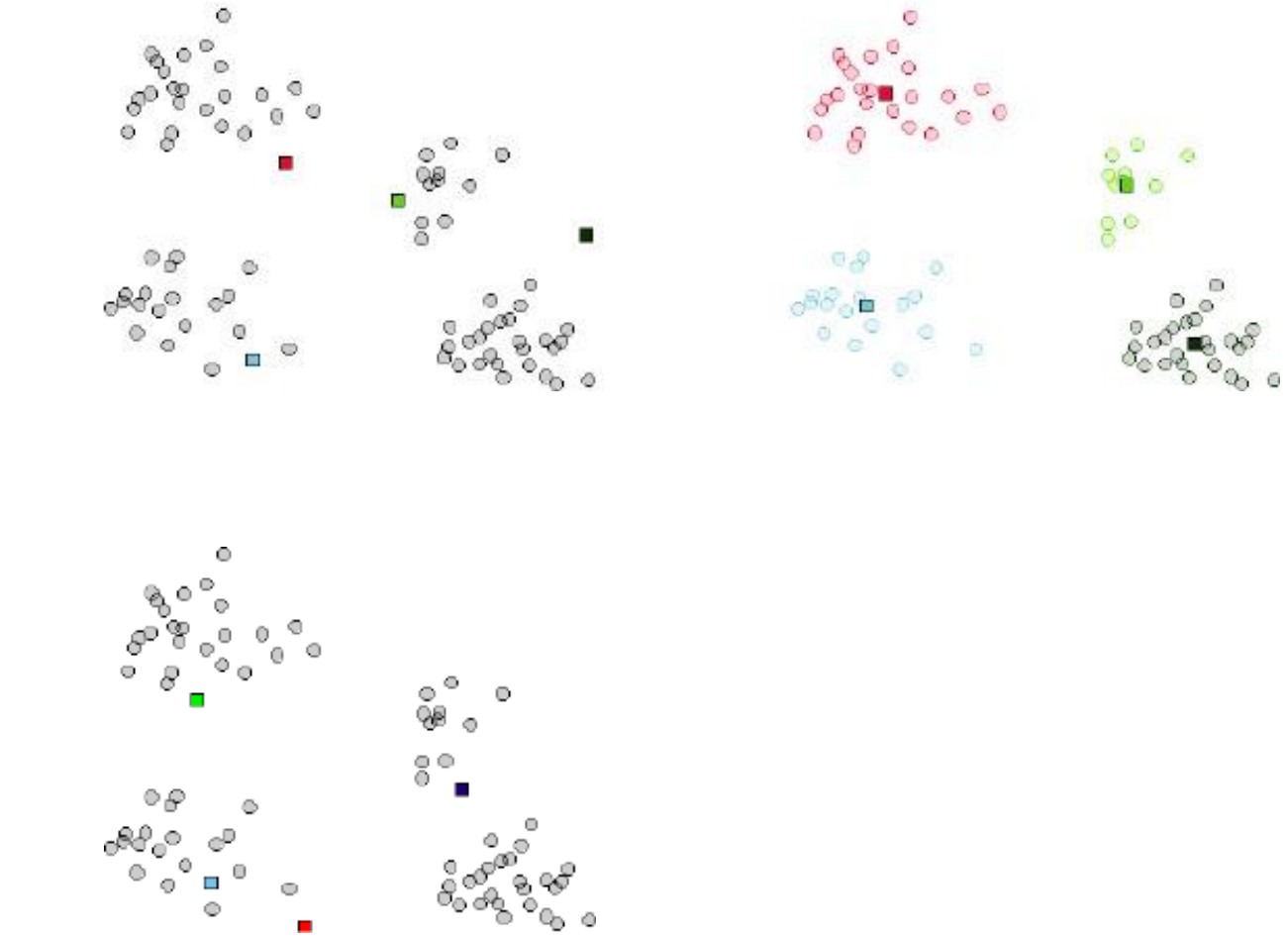
1. Initialization:
   Random generation of k cluster centers
2. Assignment:
   Assign data points to cluster centers using distance measure
3. Update the cluster centers:
   Calculation of the arithmetic mean of the related data points
4. Repeat steps 2-3 until the assignments no longer change

K-Means



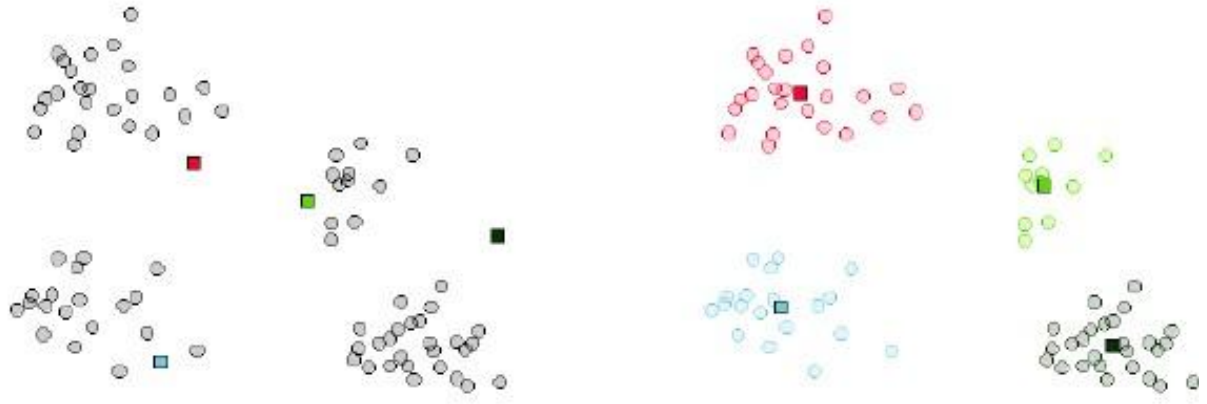
K-Means

good result

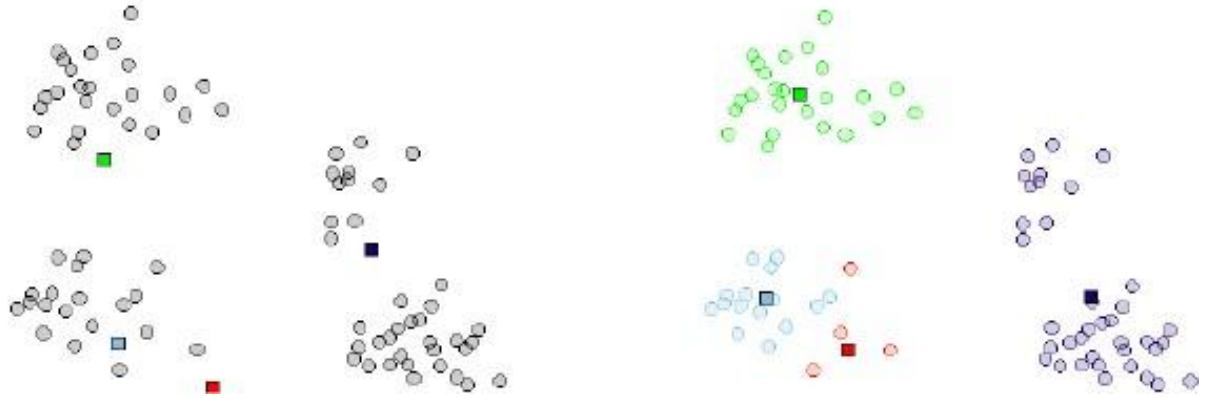


K-Means

good result



bad result



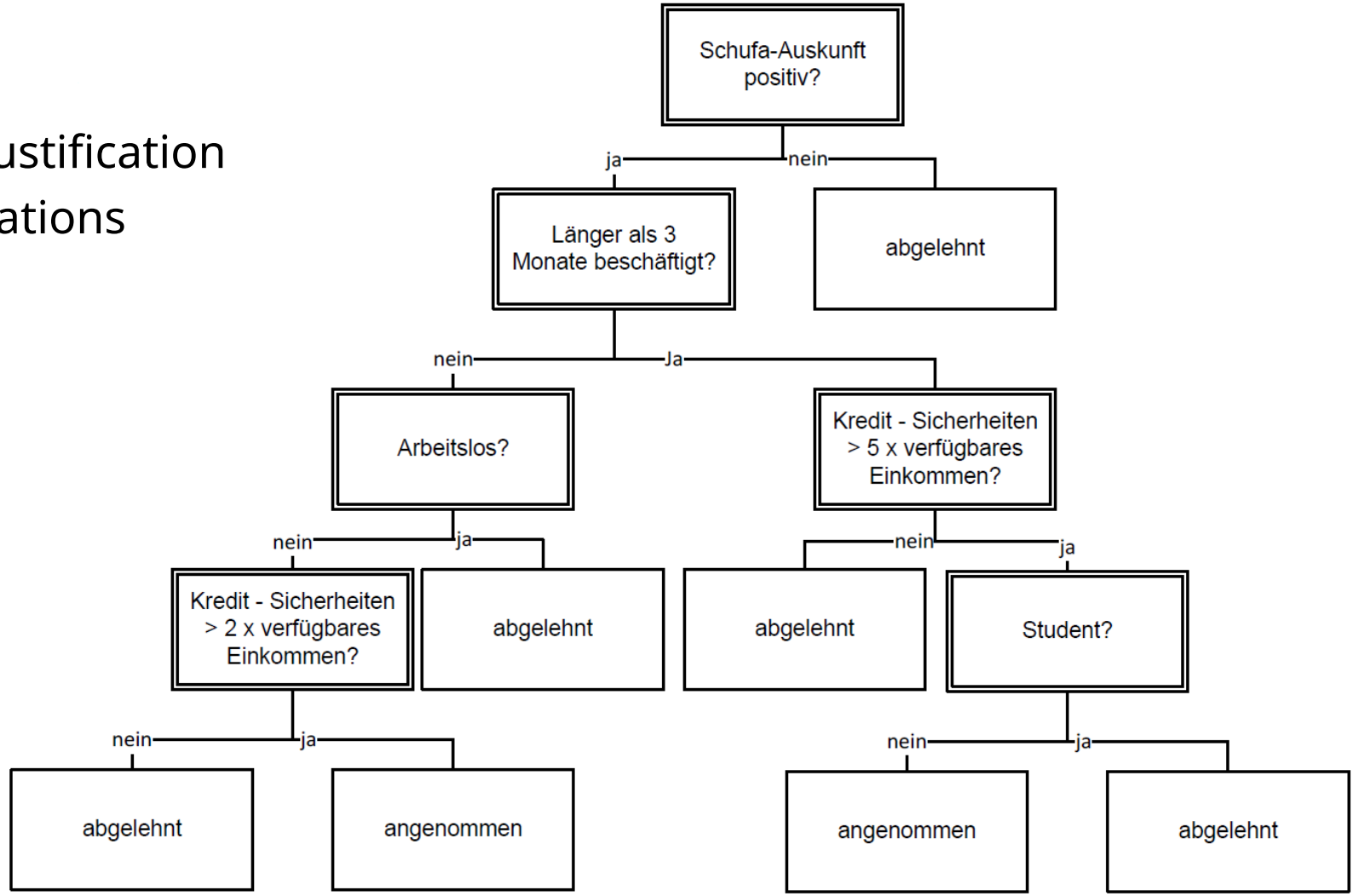
Decision Trees

Decision trees

- Decision trees classify a new example by a series of **tests** that determine the class label
- These tests are arranged in a **hierarchical structure**.
- A decision tree consists of:
  - **Nodes** (root, internal nodes)
    - Test for the value of a specific attribute
  - **Edges**
    - Corresponds to the outcome of a test
      (there is one edge for each outcome of a test)
    - Connect to the next node
  - **Leaf nodes**
    - End nodes that predict the result

Decision trees

- Easy to interpret
- Provide classification and justification
  - Required for many applications
- Learned from examples



Divide-and-conquer

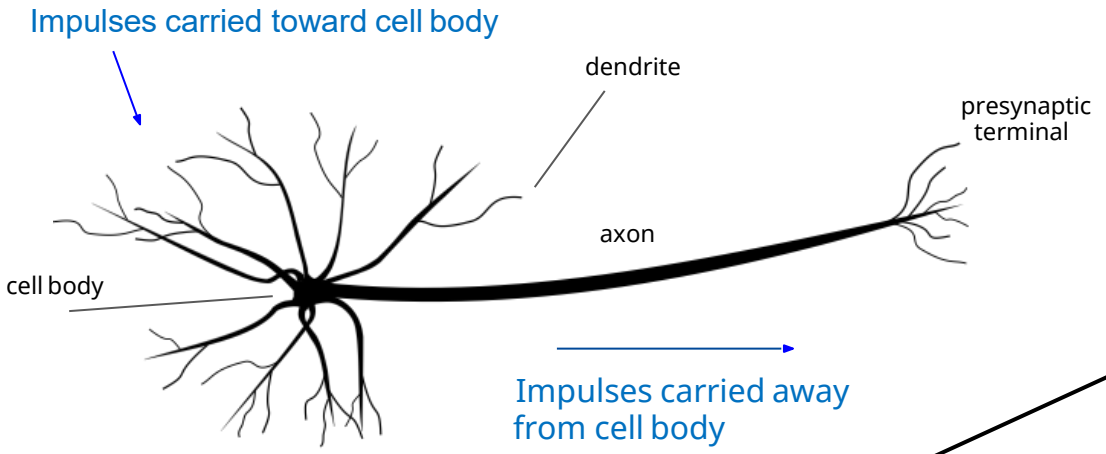
- TDIDT: Top-Down Induction of Decision Trees
- Learn trees top-down
  - Divide the problem into sub-problems
  - Solve each sub-problem

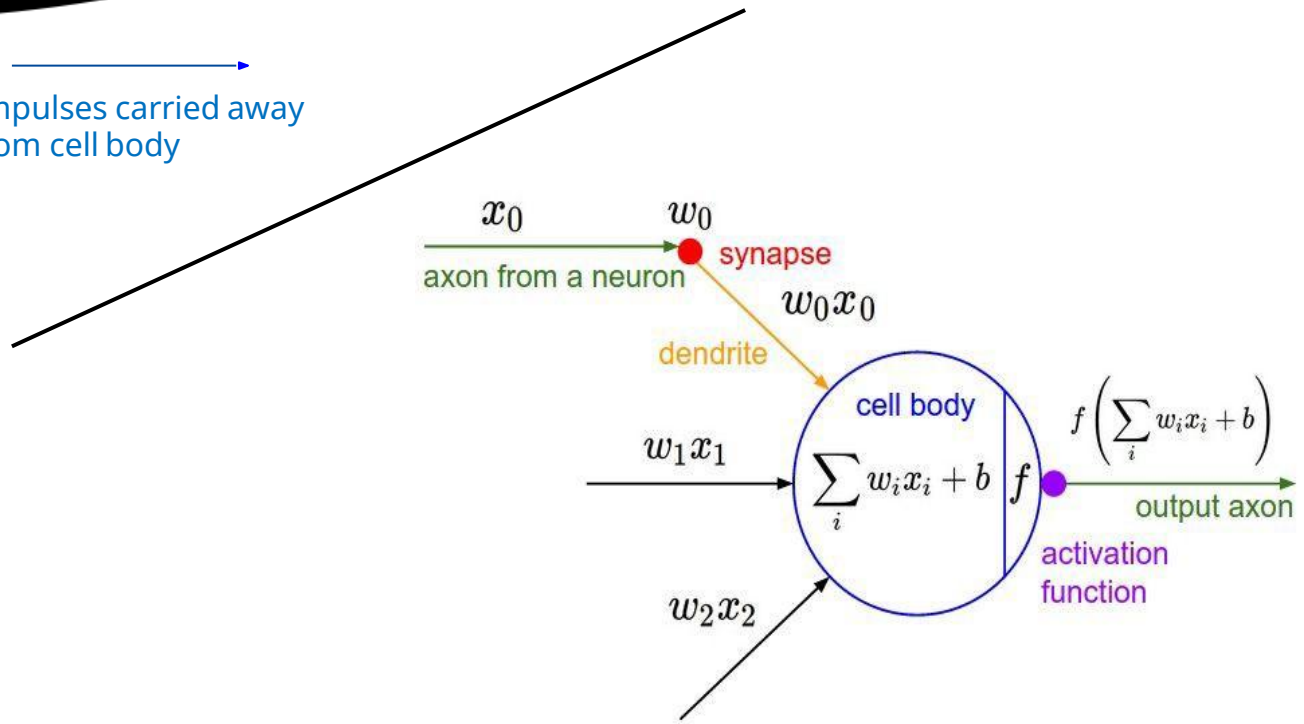
Basic algorithm:

1. Select a test (variable) for the **root node** and create a **branch** for each possible outcome (value)
2. Division of the instances into **subsets** according to the test
3. Repeat the procedure **recursively** for each branch
4. Stop the recursion if all instances of a branch have the same class

Artificial Neural Networks

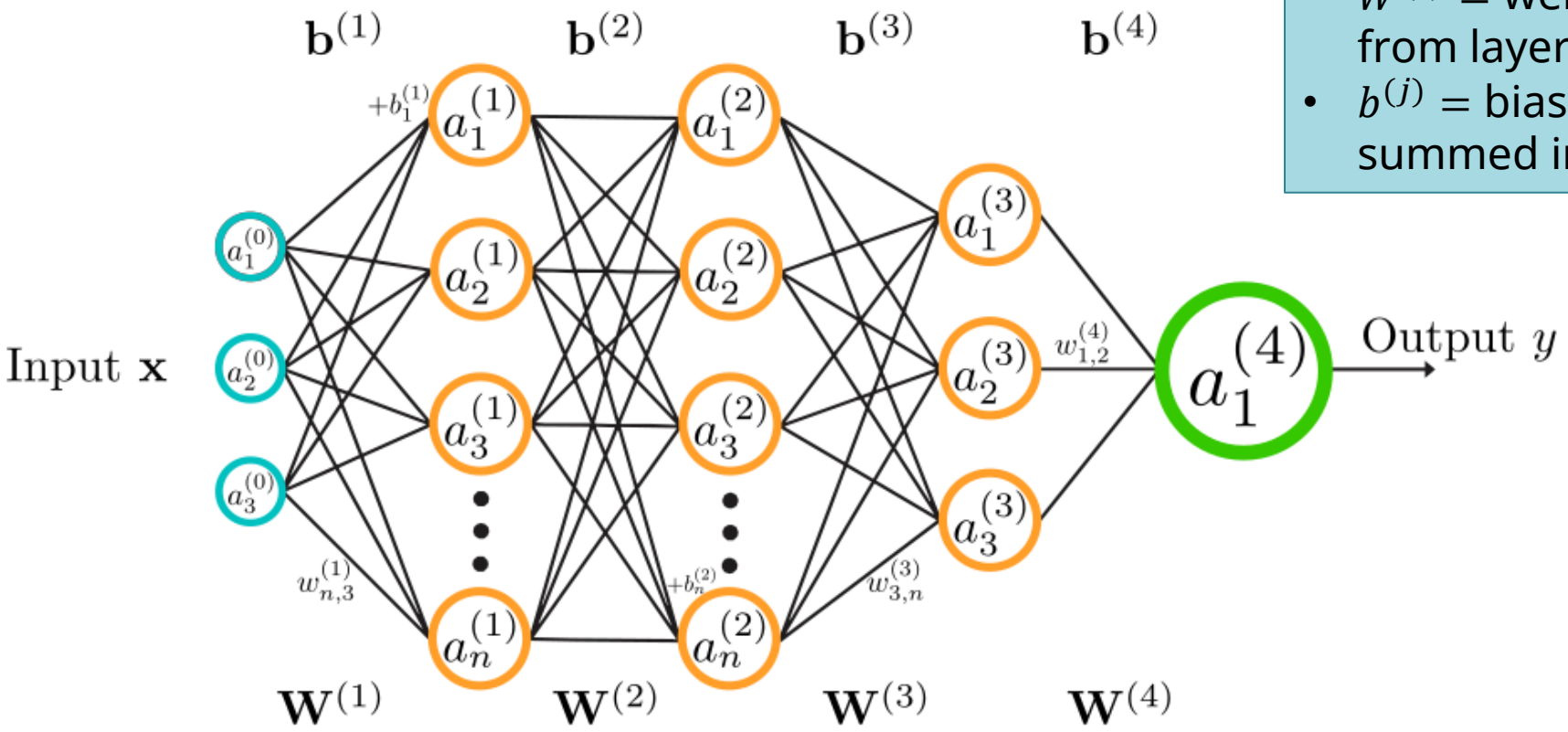
Neural Networks: Brain Stuff





Artificial Neural Network

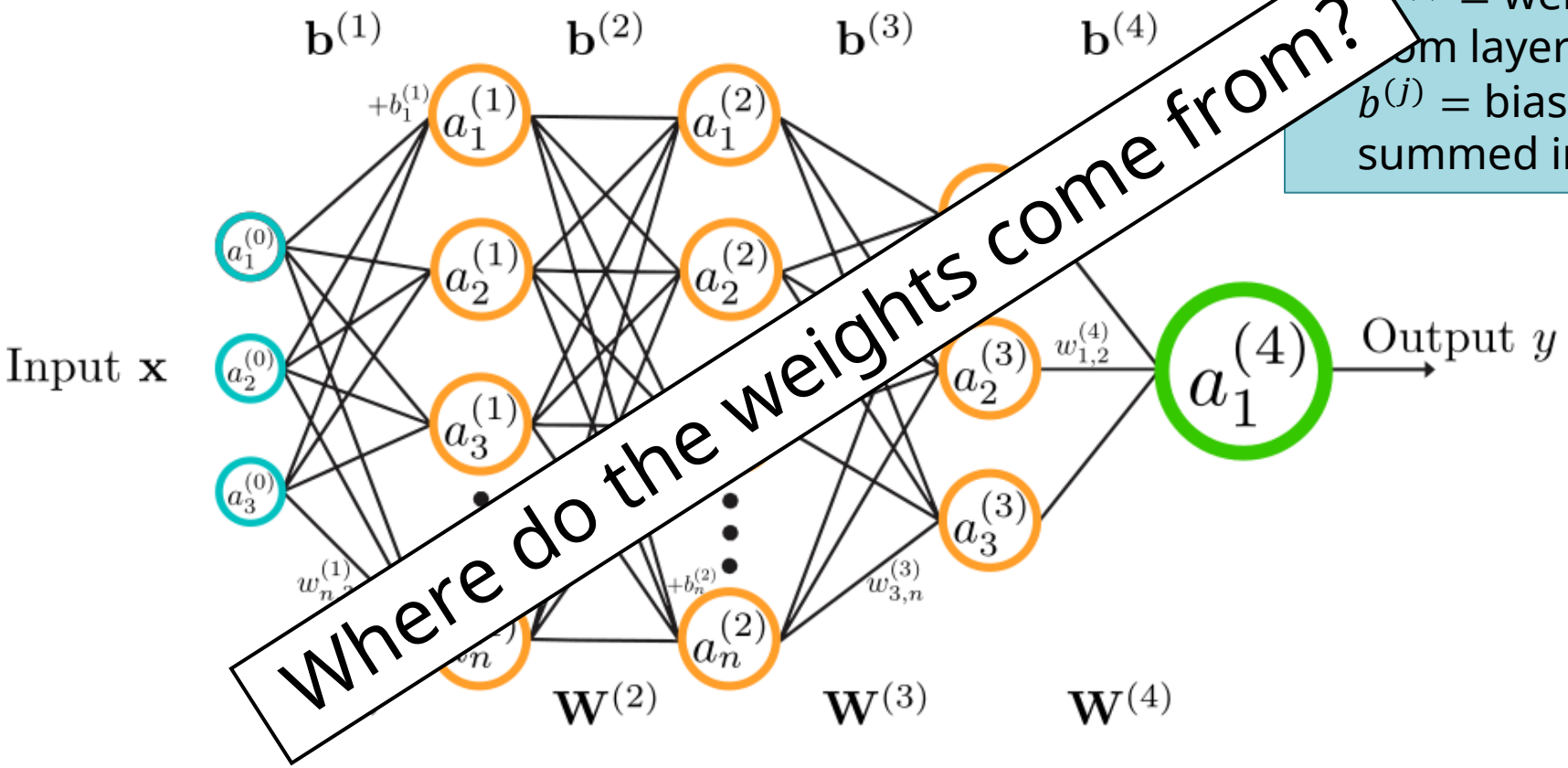
– A neural network is a directed graph with neurons as nodes and connections



- $a_i^{(j)}$ = „activation“ of neuron i in layer j
- $W^{(j)}$ = weight matrix for mapping from layer $j-1$ to layer j
- $b^{(j)}$ = bias vector added to the summed inputs of layer j

Artificial Neural Network

– A neural network is a directed graph with neurons as nodes and connections

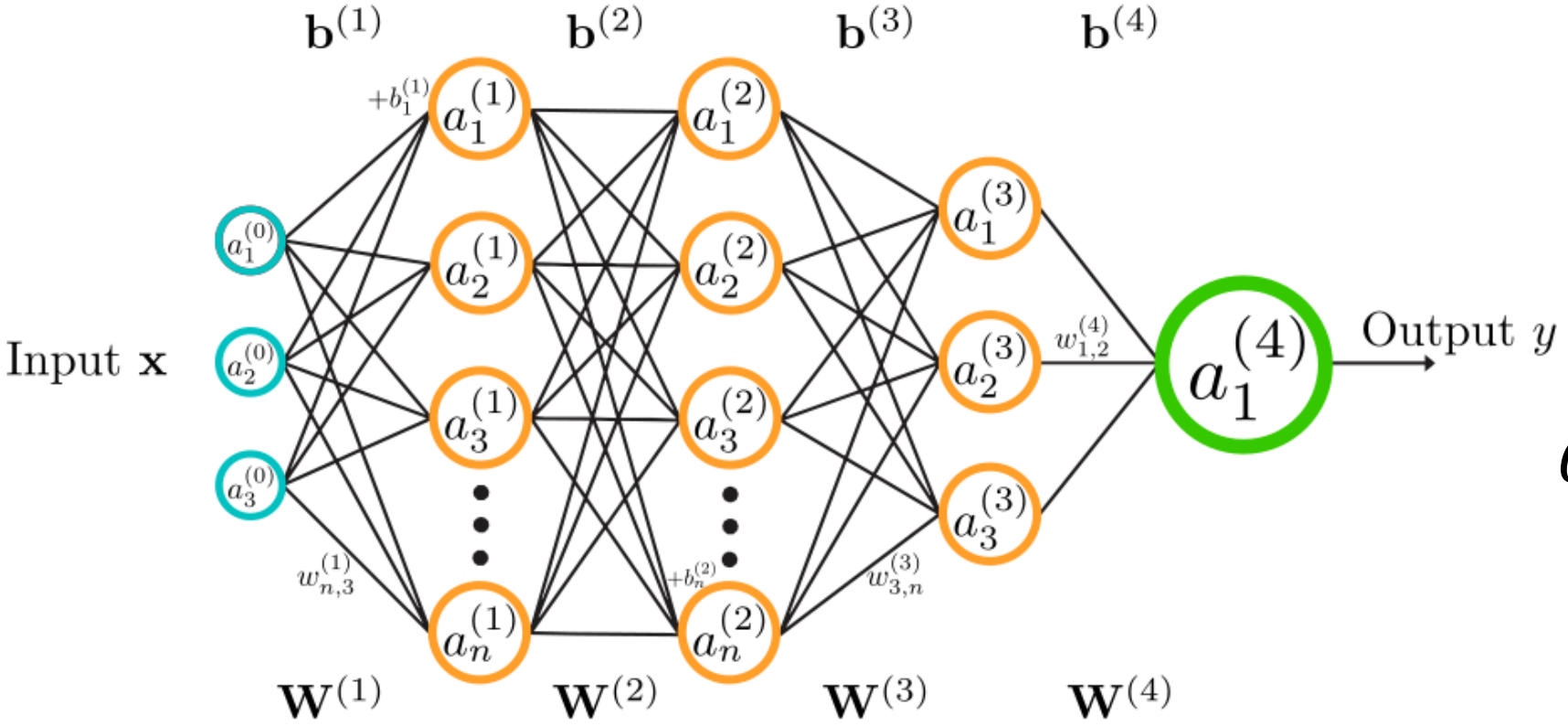


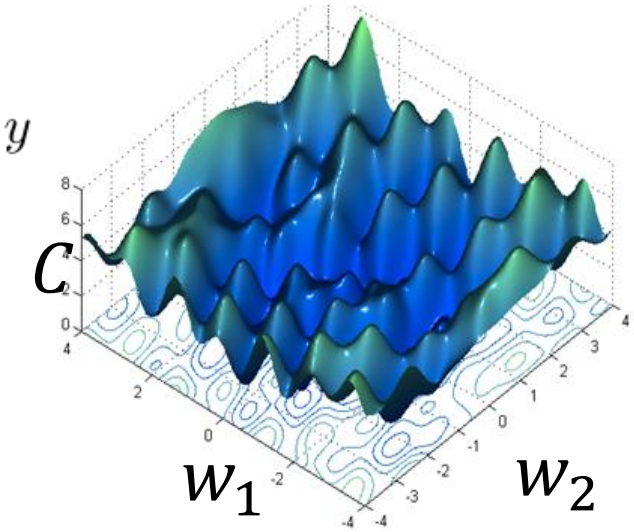
- $a_i^{(j)}$ = „activation“ of neuron i in layer j
- $w^{(j)}$ = weight matrix for mapping from layer $j-1$ to layer j
- $b^{(j)}$ = bias vector added to the summed inputs of layer j

Where do the weights come from?

Training an Artificial Neural Network

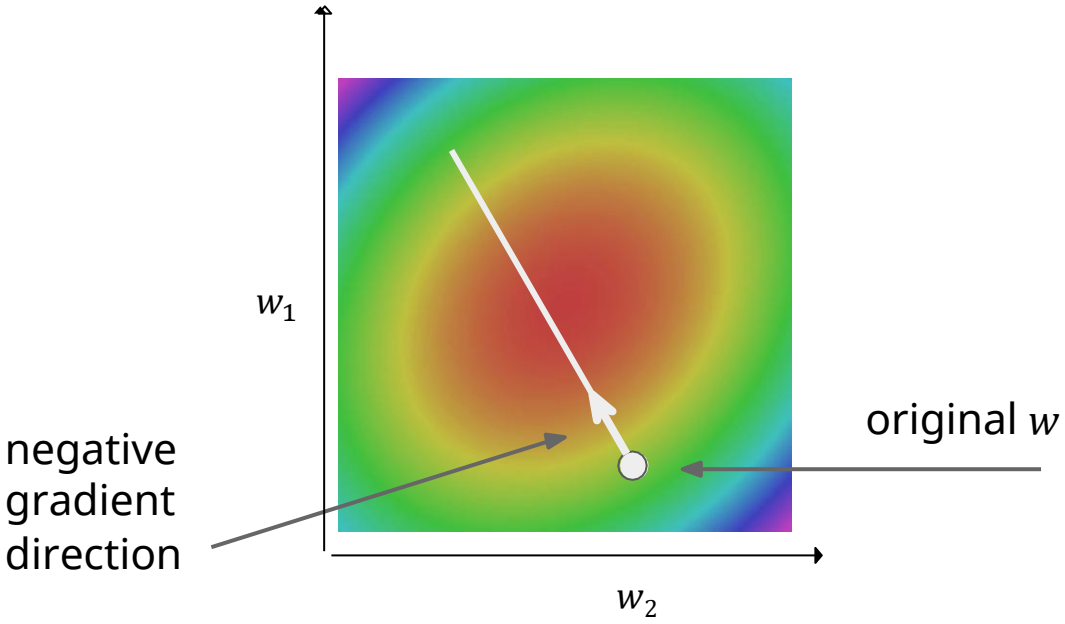
- Learning is adapting the weights so that the cost function C is minimized
  - C measures a distance between the prediction y and desired output $\hat{y}$





Training an Artificial Neural Network

- **Iterative optimization process**
  - Perform predictions for the training data
  - Calculate cost function and gradients
    - Gradients are the partial derivatives with regard to every parameter
  - Update weights in the direction of negative gradient (steepest descent)
    - Learning rate determines step size
  - Repeat until minimum is found



Foundation Models

Foundation Models

- Deep Learning Models **pretrained on enormous amounts of (synthetic) data**
- Can be applied to many tasks either directly or through **minimal finetuning**
- Costly to train, but cheap to adapt
- Pretrained models supply **priors** that own data is too small to provide
- Using pretrained foundation models requires less computation power and smaller datasets than training from scratch

Examples for Foundation Models

- BERT (Language)
- GPT (Language (+Images))
- LLAMA (Language)
- LLaVA (Vision-Language)
- CLIP (Vision-Language)
- DINO (Vision)
- Stable Diffusion (Image Generation)
- SAM (Image Segmentation)
- Whisper (Speech recognition)
- AudioLM (Audio)
- TabPFN (Tabular data)
- … and many more

Machine Learning Python Packages

Scikit Learn

Scikit Learn (Sklearn)

- Library for Machine Learning
- Efficiently implemented tools for Machine Learning and statistical modelling
- Provides methods for
  - Classification
  - Regression
  - Clustering
  - Dimension reduction
- Installation:

```
> pip install scikit-learn
```

Model Training

- **Sklearn Estimator API**
  - Step 1: Choose model class
    - Import of the respective module
  - Step 2: Set hyperparameters of the model
    - Initialize model class with desired parameters
  - Step 3: Prepare data
    - Split data in in feature matrix (X) and target vector (y).
  - Step 4: Fit the model
    - Call the `fit()` method of the model instance with the training data
  - Step 5: Apply the model
    - After training call the `predict()` method of the model instance on new data

Example classifikation

```
iris_pd = pd.read_csv('iris.csv', sep=',', header='infer')
iris_pd_train, iris_pd_test = train_test_split(iris_pd, test_size = 0.3)
```

- Step 1: Module import

```
from sklearn.neighbors import KNeighborsClassifier
```

- Step 2: Model initialization

```
classifier_knn = KNeighborsClassifier(n_neighbors = 3)
```

- Step 3: Split data

```
X_train = iris_pd_train.drop('species', axis = 1)
y_train = iris_pd_train['species']
X_test = iris_pd_test.drop('species', axis = 1)
y_test = iris_pd_test['species']
```

- Step 4: Fit the model

```
classifier_knn.fit(X_train, y_train)
```

Example classifikation (2)

– Step 4b: Evaluate the model on the test data

```
from sklearn import metrics

y_test_pred = classifier_knn.predict(X_test)
print("Test Data Accuracy:", metrics.accuracy_score(y_test, y_test_pred))
```

Calculate the accuracy metric

```
Test Data Accuracy: 0.9777777777777777
```

– Step 5: Run predictions

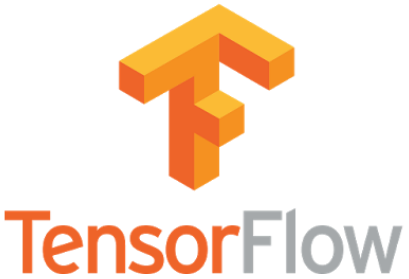
```
X_new = [[5, 5, 3, 2], [2, 4, 3, 5]]
y_new_preds = classifier_knn.predict(X_new)

for i in range(len(y_new_preds)):
    print(X_new[i], "=>", y_new_preds[i])
```

Predictions on data given as an array

```
[5, 5, 3, 2] => versicolor
[2, 4, 3, 5] => virginica
```

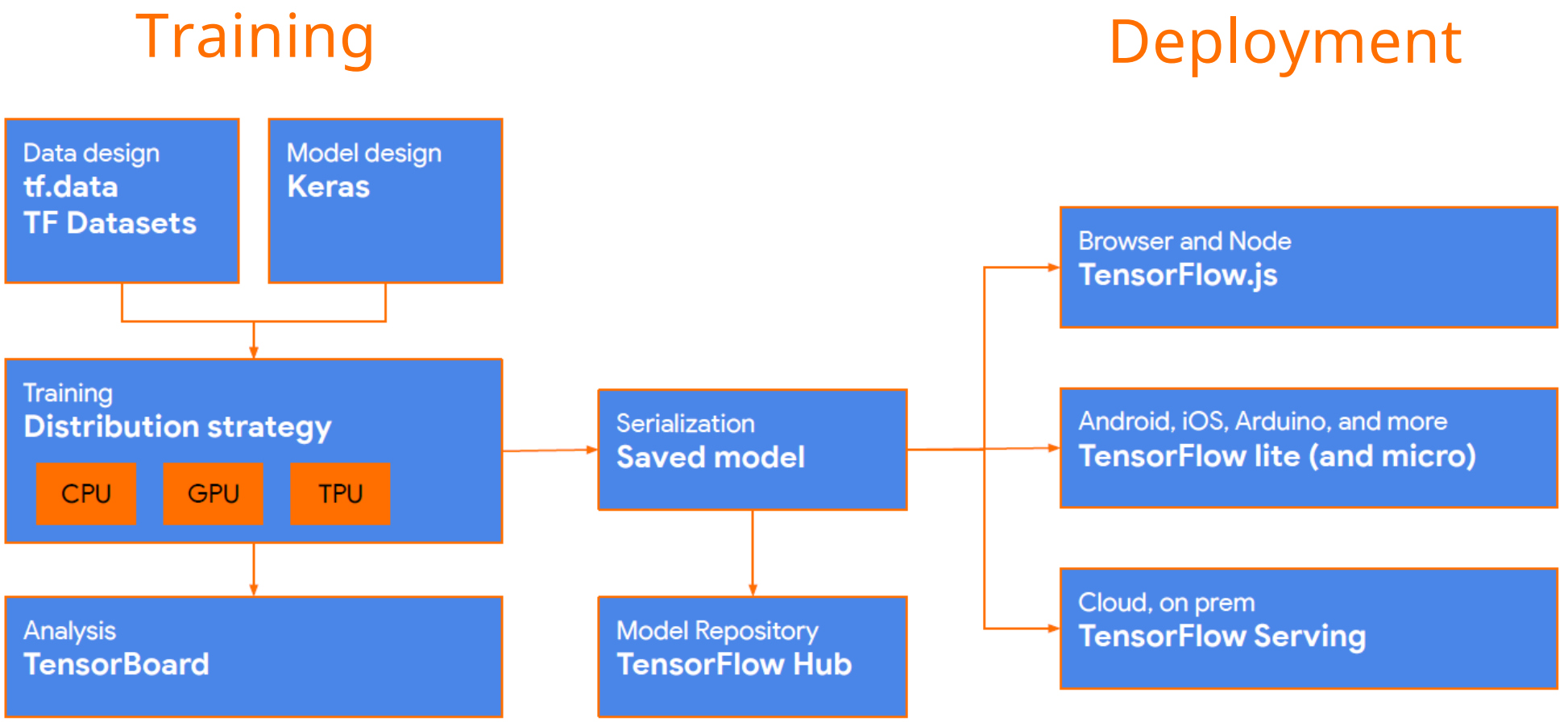
TensorFlow / Keras

TensorFlow

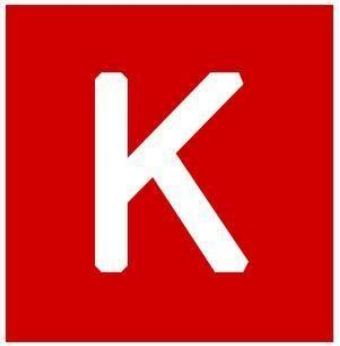


- URL: https://www.tensorflow.org
- Originally developed to conduct machine learning and neural network research
- Software library for numerical computation using data flow graphs
- High flexibility + scalability

Tensorflow Platform



Keras



- https://keras.io/
- Minimalist, highly modular neural networks library
- Written in Python
- Capable of running on top of TensorFlow, PyTorch, JAX
  - Built-in to TF2
- Developed with a focus on enabling fast experimentation

Keras Models

- The core data structure of Keras is a model
  - Models are a way to organize layers

```
model = keras.models.Sequential([
    keras.layers.Flatten(),
    keras.layers.Dense(512, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparese_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)

model.evaluate(x_test, y_test)
```

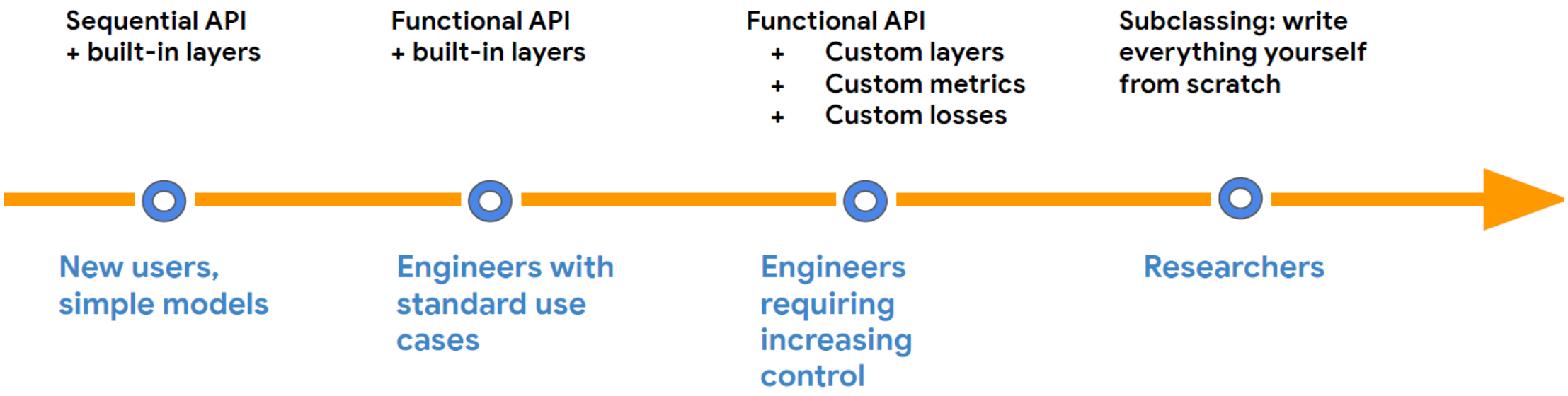
Define layer structure

Prepare model for training

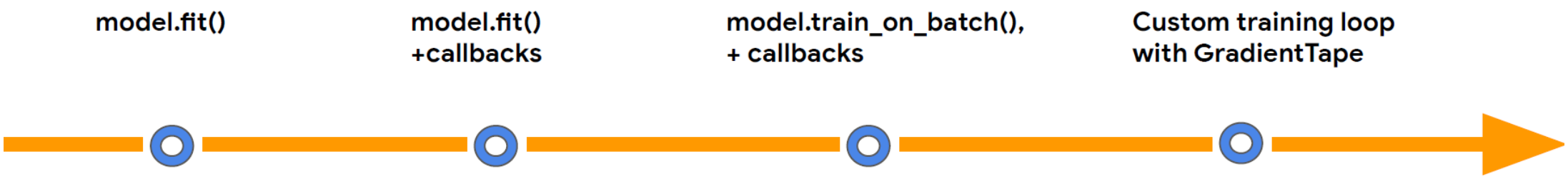
Train model

Evaluate the trained model

Model building
(from simple to arbitrary flexible)



Model training
(from simple to arbitrary complex)



Quick experiment

Customize your training loop
Add checkpointing, early stopping, TensorBoard monitoring, send Slack notifications...

Custom training loop using built-in optimizers and losses
e.g. GANs

Complete control
e.g. new optimization algorithm; easily modify gradients as you go.

Next Steps

Next Steps

- Download the additional files from https://ilias.uni-marburg.de/goto.php/fold/4776269
- Navigate to the folder containing the downloaded files in a terminal
- Start jupyter lab in this folder

```
> jupyter lab
```

- Open http://localhost:8888 in a web browser if it does not open automatically
- The tasks for the first day are found in the notebook **`ProPra_AI_day2.ipynb`**
- Have fun!

- If you run into problems, ask the tutors for help